Task 1

1.a. Write the **Pokemon** class so that the following code generates the output below:

Driver Code	Output

public class PokemonTester{
  public static void main(String[] args){
  Pokemon pikachu = new Pokemon("Pikachu");
  pikachu.attack();
  pikachu.attack("Thunderbolt");
  pikachu.attack("Iron Tail", 90);
  }
}
	Pikachu attacks with a basic move!
	Pikachu uses Thunderbolt!
	Pokemon uses Iron Tail with power 90!

1.b. What type of polymorphism is depicted in the code above?

এই টাস্কটি সমাধান করার জন্য আমাদের **Method Overloading** ব্যবহার করতে হবে, কারণ এখানে `attack()` মেথডটি ভিন্ন ভিন্ন প্যারামিটার নিয়ে তিনবার কল করা হয়েছে।

Java
```
public class Pokemon {
    public String name;

    // কনস্ট্রাক্টর
    public Pokemon(String name) {
        this.name = name;
    }

    // ১. কোনো প্যারামিটার ছাড়া attack মেথড
    public void attack() {
        System.out.println(this.name + " attacks with a basic move!");
    }

    // ২. শুধু একটি String (move) প্যারামিটার সহ attack মেথড
    public void attack(String move) {
        System.out.println(this.name + " uses " + move + "!");
    }

    // ৩. একটি String এবং একটি int (power) প্যারামিটার সহ attack মেথড
    public void attack(String move, int power) {
        // আউটপুট অনুযায়ী এখানে "Pokemon" শব্দটি ব্যবহার করা হয়েছে
        System.out.println("Pokemon uses " + move + " with power " + power + "!");
    }
}
```

সহজ ব্যাখ্যা **(Bangla Explanation)**

এখানে মূলত একটি ক্লাসের ভেতর একই নামে একাধিক মেথড তৈরি করা হয়েছে, যেটাকে আমরা প্রোগ্রামিংয়ের ভাষায় **Method Overloading** বলি।

1. **Constructor:** `Pokemon pikachu = new Pokemon("Pikachu");` লাইনের মাধ্যমে যখন অবজেক্ট তৈরি হয়, তখন `Pikachu` নামটি সেট হয়ে যায়।
2. প্রথম **attack():** যখন কোনো আর্গুমেন্ট ছাড়া কল করা হয়, তখন এটি "basic move" প্রিন্ট করে।
3. দ্বিতীয় **attack("Thunderbolt"):** এখানে একটি String পাস করা হয়েছে, তাই এটি `move` হিসেবে "Thunderbolt" গ্রহণ করে প্রিন্ট করে।
4. তৃতীয় **attack("Iron Tail", 90):** এখানে একটি নাম এবং একটি সংখ্যা (power) পাঠানো হয়েছে। আপনার দেওয়া আউটপুট লক্ষ্য করলে দেখবেন এখানে নামের জায়গায় "Pokemon" ব্যবহার করা হয়েছে, তাই আমরা কোডে সরাসরি ওই শব্দটি প্রিন্ট করেছি।

Task 1.b: Polymorphism Type

Question: What type of polymorphism is depicted in the code above?

Answer: এটি **Compile-time Polymorphism** (যাকে Static Polymorphism-ও বলা হয়)।

ব্যাখ্যা**:** যেহেতু এখানে **Method Overloading** ব্যবহার করা হয়েছে এবং কম্পাইলার কোড রান করার আগেই জানে কোন মেথডটি কল হবে, তাই এটি কম্পাইল-টাইম পলিমরফিজম।

==========

Task 2

Write the **UsedBattery** and the **PlasticBottle** classes derived from Waste class so that the following code generates the output below.

You also need to **complete the SorterBot class** for this code to work. You cannot create any additional methods in the **SorterBot** class.

Tester

```
public class RecyclingPlant {
 public static void main(String[] args) {
  SorterBot robot = new SorterBot();
  UsedBattery bat1 = new UsedBattery("Duracell-X", true);
  PlasticBottle bot1 = new PlasticBottle("Coke-Zero", 500);
  Waste unknown = new Waste("Mystery-Box");
  System.out.println("######################");
  robot.processItem(bat1);
  robot.processItem(bot1);
  robot.processItem(unknown);
  robot.processItem(new UsedBattery("Energizer", false));
 }
}
//Parent Class
class Waste {
 String id;

 Waste(String id) {
  this.id = id;
 }
}
//Disjoint Class
class SorterBot {
 public void processItem(Waste item) {
  //WRITE YOUR CODE HERE
  System.out.println("------------------------------");
 }
}
```

Output

```
######################
Scanning Item Duracell-X [Type: BATTERY]
Duracell-X is leaking. Sealing in concrete
container.
------------------------------
Scanning Item Coke-Zero [Type: PLASTIC]
Coke-Zero compressed from 500ml to flat
disk.
------------------------------
Scanning Item Mystery-Box [Type: UNKNOWN]
Item sent to generic incinerator.
------------------------------
Scanning Item Energizer [Type: BATTERY]
Energizer stored in dry cell.
------------------------------
```

এই সমস্যাটি সমাধান করার জন্য আমাদের **Inheritance** (উত্তরাধিকার) এবং **instanceof** অপারেটর ব্যবহার করতে হবে। যেহেতু `SorterBot` এর `processItem` মেথডটি একটি `Waste` টাইপ প্যারামিটার গ্রহণ করে, তাই আমাদের চেক করতে হবে সেটি আসলে কোন ধরনের অবজেক্ট।

নিচে জাভা কোড এবং তার বাংলা ব্যাখ্যা দেওয়া হলো:

Task 2: Solution Code

Java

```
// UsedBattery class (Derived from Waste)
class UsedBattery extends Waste {
    boolean isLeaking;

    UsedBattery(String id, boolean isLeaking) {
        super(id);
        this.isLeaking = isLeaking;
    }
```

```
}

// PlasticBottle class (Derived from Waste)
class PlasticBottle extends Waste {
    int volume;

    PlasticBottle(String id, int volume) {
        super(id);
        this.volume = volume;
    }
}

// SorterBot class logic
class SorterBot {
    public void processItem(Waste item) {
        // instanceof ব্যবহার করে অবজেক্টের ধরণ চেক করা হচ্ছে
        if (item instanceof UsedBattery) {
            UsedBattery b = (UsedBattery) item; // Downcasting
            System.out.println("Scanning Item " + b.id + " [Type: BATTERY]");
            if (b.isLeaking) {
                System.out.println(b.id + " is leaking. Sealing in concrete container.");
            } else {
                System.out.println(b.id + " stored in dry cell.");
            }
        }
        else if (item instanceof PlasticBottle) {
            PlasticBottle p = (PlasticBottle) item; // Downcasting
            System.out.println("Scanning Item " + p.id + " [Type: PLASTIC]");
            System.out.println(p.id + " compressed from " + p.volume + "ml to flat disk.");
        }
        else {
            System.out.println("Scanning Item " + item.id + " [Type: UNKNOWN]");
            System.out.println("Item sent to generic incinerator.");
        }

        System.out.println("-----------------------------");
    }
}
```

সহজ ব্যাখ্যা **(Bangla Explanation)**

১. **Inheritance (**উত্তরাধিকার**):** `UsedBattery` এবং `PlasticBottle` ক্লাস দুটিকে `Waste` ক্লাসের সাথে `extends` কিওয়ার্ড দিয়ে কানেক্ট করা হয়েছে। `super(id)` ব্যবহার করা হয়েছে যাতে চাইল্ড ক্লাস থেকে প্যারেন্ট ক্লাসের কনস্ট্রাক্টর কল করে `id` সেট করা যায়।

২. **SorterBot** এর **logic:**

- এখানে `processItem(Waste item)` মেথডটি `Waste` এর যেকোনো চাইল্ড অবজেক্টকে গ্রহণ করতে পারে। এটাকে বলা হয় **Polymorphism**।
- **instanceof:** এই কিওয়ার্ডটি দিয়ে আমরা চেক করেছি `item` টা কি `UsedBattery` নাকি `PlasticBottle`?
- **Downcasting:** যখন আমরা নিশ্চিত হই যে `item` টি একটি নির্দিষ্ট ক্লাসের (যেমন `UsedBattery`), তখন আমরা `(UsedBattery) item` লিখে তাকে নির্দিষ্ট টাইপে রূপান্তর করি যাতে ওই ক্লাসের বিশেষ ভেরিয়েবল (যেমন `isLeaking` বা `volume`) অ্যাক্সেস করা যায়।

৩. **Output Match:** * যদি ব্যাটারি লিক করে (`true`), তবে কংক্রিট কন্টেইনারে রাখার মেসেজ আসবে।

- যদি প্লাস্টিক বোতল হয়, তবে সেটির ভলিউম প্রিন্ট হবে।
- যদি কোনোটিই না হয় (যেমন `Mystery-Box`), তবে সেটি `else` ব্লকে গিয়ে `UNKNOWN` হিসেবে ইনসিনারেটরে যাবে।

==========

Task 3

Your task is to design the **StudyRoom** class with appropriate variables and methods such that the following tester code produces the expected output. Note:

- Assume that each **StudyRoom** can add two books.
- You cannot use any arrays in the **StudyRoom** class.
- You should use the given **Library** and **SilentStudyRoom** classes' variables and methods as needed.
- You cannot modify the given **Library** and **SilentStudyRoom** classes.

Tester Code	Expected Output

`public class TestLibrary{`
`  public static void main(String[] args) {`
`  Library library = new Library("The Mind Maze");`
`  library.showRoomInfo();`
`  System.out.println("======== 1 ========");`
`  StudyRoom room9 = new StudyRoom("Study Hub");`
`  SilentStudyRoom room9A  = new SilentStudyRoom("Focus Room");`
`  room9.addBook("Data Structures");`
`  System.out.println("======== 2 ========");`
`  room9.addBook("Operating Systems");`
`  room9.showRoomInfo();`
`  System.out.println("======== 3 ========");`
`  System.out.println("Total books: " +StudyRoom.totalBooks);`
`  System.out.println("======== Add Book ========");`
`  room9A.addBook(library, room9);`
`  System.out.println("======== 4 ========");`
`  room9A.showRoomInfo();`
`  System.out.println("Total books: " +StudyRoom.totalBooks);`
`  System.out.println("======== Add Book Again ========");`
`  room9A.addBook(library, room9);`
`  System.out.println("======== 5 ========");`
`  room9A.showRoomInfo();`
`  System.out.println("Total books: " + StudyRoom.totalBooks);`
`  }`
`}`
	`Library Name: The Mind Maze`
	`======== 1 ========`
	`======== 2 ========`
	`Study Hub Details:`
	`Book 1: Data Structures`
	`Book 2: Operating Systems`
	`======== 3 ========`
	`Total books: 2`
	`======== Add Book ========`
	`Library Name: The Mind Maze`
	`Adding book: Data Structures`
	`Book 1 Removed from Study Hub`
	`Book added successfully!`
	`======== 4 ========`
	`Focus Room Details:`
	`Book 1: Data Structures`
	`No book added`
	`Total books: 1`
	`======== Add Book Again ========`
	`Library Name: The Mind Maze`
	`Adding book: Operating Systems`
	`Book 2 Removed from Study Hub`
	`Book added successfully!`
	`======== 5 ========`
	`Focus Room Details:`
	`Book 1: Data Structures`
	`Book 2: Operating Systems`
	`Total books: 0`

এই টাস্কটি সমাধান করার জন্য আমাদের `StudyRoom` ক্লাসটি এমনভাবে ডিজাইন করতে হবে যা `Library` ক্লাসকে ইনহেরিট (Inherit) করে। এখানে অ্যারে ব্যবহার করা নিষিদ্ধ, তাই আমরা আলাদা দুটি স্ট্রিং ভেরিয়েবল ব্যবহার করব বই সংরক্ষণের জন্য। এছাড়া, বইয়ের সংখ্যা ট্র্যাকিং করার জন্য একটি **static** ভেরিয়েবল ব্যবহার করতে হবে।

নিচে জাভা কোড এবং এর সহজ ব্যাখ্যা দেওয়া হলো:

Task 3: StudyRoom Class Solution

Java

```
class StudyRoom extends Library {
    public String book1;
    public String book2;
    public static int totalBooks = 0; // স্ট্যাটিক ভেরিয়েবল যা সব অবজেক্ট শেয়ার করবে
```

```
public StudyRoom(String name) {
    super(name);
}

// বই যুক্ত করার মেথড
public void addBook(String bookName) {
    if (book1 == null) {
        book1 = bookName;
        totalBooks++;
    } else if (book2 == null) {
        book2 = bookName;
        totalBooks++;
    }
}

// SilentStudyRoom থেকে বইয়ের নাম পেতে এই গেটার (Getter) মেথডগুলো প্রয়োজন
public String getBook1() {
    return book1;
}

public String getBook2() {
    return book2;
}

// SilentStudyRoom এ বই সেট করতে এই সেটার (Setter) মেথডগুলো প্রয়োজন
public void setBook1(String bookName) {
    this.book1 = bookName;
}

public void setBook2(String bookName) {
    this.book2 = bookName;
}

// বই রিমুভ করার মেথড এবং টোটাল কাউন্ট কমানো
public void removeBook(int bookNumber) {
    if (bookNumber == 1) {
        System.out.println("Book 1 Removed from " + this.name);
        book1 = null;
        totalBooks--;
    } else if (bookNumber == 2) {
        System.out.println("Book 2 Removed from " + this.name);
        book2 = null;
        totalBooks--;
```

```
        }
    }

    @Override
    public void showRoomInfo() {
        System.out.println(this.name + " Details:");
        System.out.println("Book 1: " + (book1 == null ? "No book added" : book1));
        System.out.println("Book 2: " + (book2 == null ? "No book added" : book2));
    }
}
```

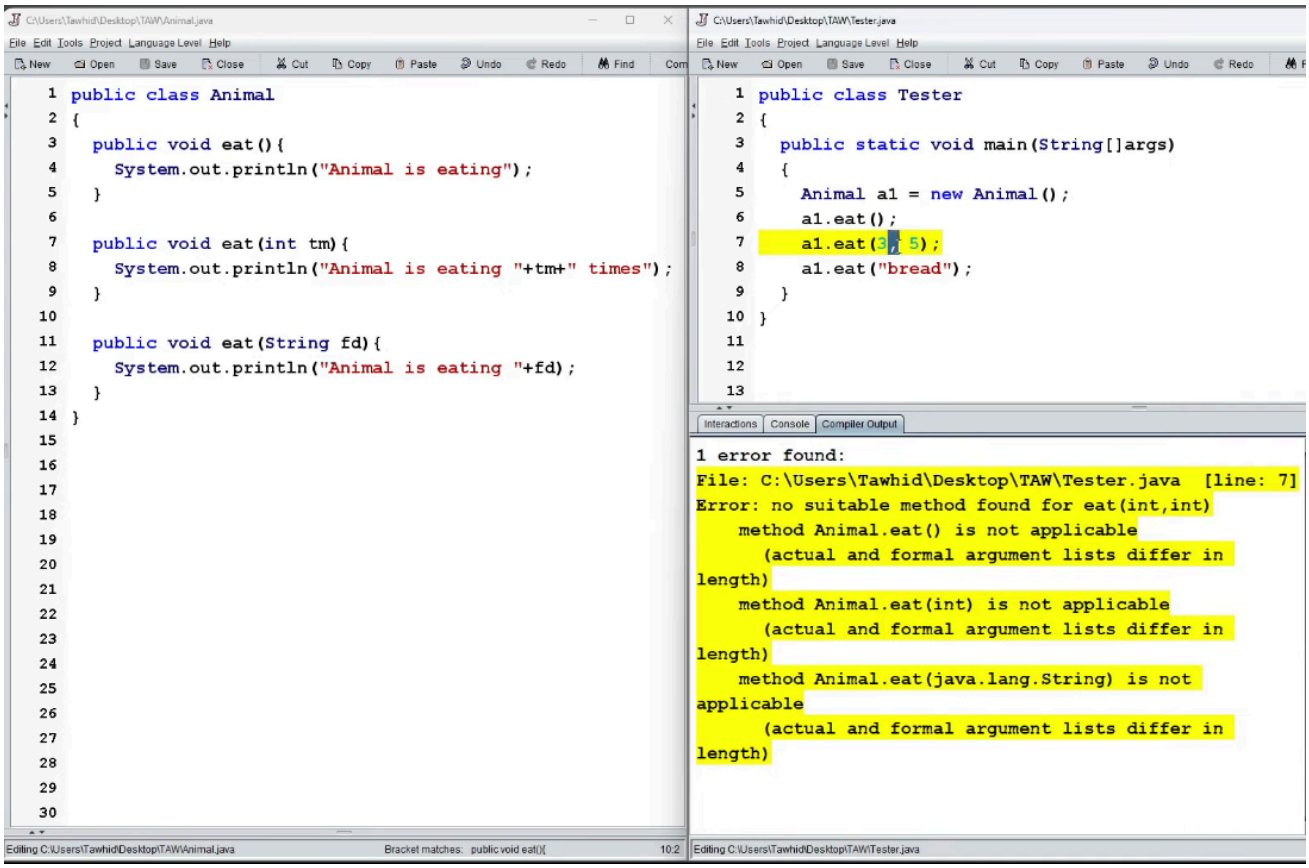
সহজ ব্যাখ্যা **(Bangla Explanation)**

১. **Static Variable (`totalBooks`):** এখানে `totalBooks` কে `static` হিসেবে ঘোষণা করা হয়েছে কারণ সব রুমের (StudyRoom এবং SilentStudyRoom) মোট বইয়ের সংখ্যা একটি কমন ভেরিয়েবলে থাকা দরকার। যখনই কোনো বই যোগ হয় এটি বাড়ে, আর `removeBook` হলে এটি কমে যায়।

২. **Encapsulation & Accessors:** `SilentStudyRoom` ক্লাসের কোড লক্ষ্য করলে দেখবেন সেখানে `room.getBook1()` এবং `room.setBook1()` মেথডগুলো কল করা হয়েছে। তাই আমরা `StudyRoom` ক্লাসে এই গেটার (Getter) এবং সেটার (Setter) মেথডগুলো তৈরি করেছি যাতে অন্য ক্লাস থেকে বইয়ের ডেটা অ্যাক্সেস করা যায়।

৩. **`showRoomInfo` Method:** এই মেথডটি ওভাররাইড (Override) করা হয়েছে যাতে আউটপুট ফরম্যাট অনুযায়ী রুমের নাম এবং বইয়ের তালিকা দেখানো যায়। বই না থাকলে "No book added" প্রিন্ট করার জন্য আমরা টারনারি অপারেটর ব্যবহার করেছি।

৪. **Logic Flow:** যখন `room9A.addBook(library, room9)` কল হয়, তখন এটি `room9` থেকে বই নিয়ে নিজের রুমে সেট করে এবং `room9` থেকে সেটি রিমুভ করে দেয়।



C:\Users\Tawhid\Desktop\TAW\Animal.java

File Edit Tools Project Language Level Help

New Open Save Close Cut Copy Paste Undo Redo Find

```
1  public class Animal
2  {
3    public void eat(){
4      System.out.println("Animal is eating");
5    }
6
7    public void eat(int tm){
8      System.out.println("Animal is eating "+tm+" times");
9    }
10
11   public void eat(String fd){
12     System.out.println("Animal is eating "+fd);
13   }
14 }
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
```

Editing C:\Users\Tawhid\Desktop\TAW\Animal.java Bracket matches: public void eat(){ 10:2

C:\Users\Tawhid\Desktop\TAW\Tester.java

File Edit Tools Project Language Level Help

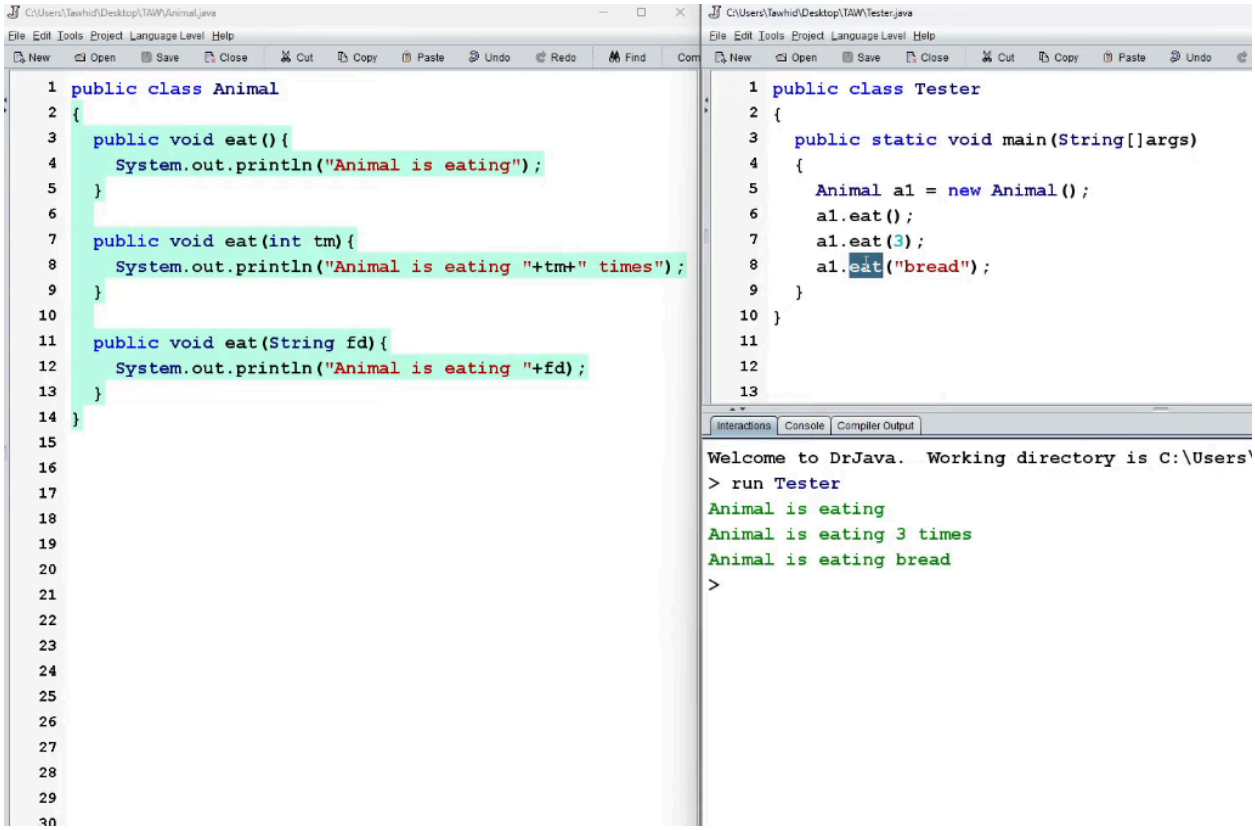
New Open Save Close Cut Copy Paste Undo Redo

```
1  public class Tester
2  {
3    public static void main(String[]args)
4    {
5      Animal a1 = new Animal();
6      a1.eat();
7      a1.eat(3, 5);
8      a1.eat("bread");
9    }
10 }
11
12
13
```

Interactions | Console | Compiler Output

```
1 error found:
File: C:\Users\Tawhid\Desktop\TAW\Tester.java  [line: 7]
Error: no suitable method found for eat(int,int)
    method Animal.eat() is not applicable
      (actual and formal argument lists differ in
length)
    method Animal.eat(int) is not applicable
      (actual and formal argument lists differ in
length)
    method Animal.eat(java.lang.String) is not
applicable
      (actual and formal argument lists differ in
length)
```

Editing C:\Users\Tawhid\Desktop\TAW\Tester.java



C:\Users\Tawhid\Desktop\TAW\Animal.java

File Edit Tools Project Language Level Help

New Open Save Close Cut Copy Paste Undo Redo Find

```
1  public class Animal
2  {
3    public void eat(){
4      System.out.println("Animal is eating");
5    }
6
7    public void eat(int tm){
8      System.out.println("Animal is eating "+tm+" times");
9    }
10
11   public void eat(String fd){
12     System.out.println("Animal is eating "+fd);
13   }
14 }
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
```

C:\Users\Tawhid\Desktop\TAW\Tester.java

File Edit Tools Project Language Level Help

New Open Save Close Cut Copy Paste Undo

```
1  public class Tester
2  {
3    public static void main(String[]args)
4    {
5      Animal a1 = new Animal();
6      a1.eat();
7      a1.eat(3);
8      a1.eat("bread");
9    }
10 }
11
12
13
```

Interactions Console Compiler Output

```
Welcome to DrJava.  Working directory is C:\Users\
> run Tester
Animal is eating
Animal is eating 3 times
Animal is eating bread
>
```